

Modèle de données JSON

Ce document présente la structure JSON de chacune des trois collections MongoDB utilisées par l'application : `users`, `articles` et `comments`. Pour chaque collection, un tableau détaille les champs (nom, type BSON, contraintes, description) suivi d'un exemple de document réel tel qu'il serait stocké en base.

1. Collection : `users`

Représente un compte utilisateur, avec un rôle unique parmi `ADMIN`, `AUTHOR` ou `READER`.

Champ	Type BSON	Contrainte	Description
<code>_id</code>	<code>ObjectId</code>	Généré automatiquement	Identifiant unique MongoDB du document
<code>fullName</code>	<code>String</code>	Requis	Nom complet de l'utilisateur
<code>email</code>	<code>String</code>	Requis, unique (index)	Adresse email, utilisée pour l'authentification
<code>password</code>	<code>String</code>	Requis	Mot de passe hashé (BCrypt), jamais stocké en clair
<code>role</code>	<code>String (enum)</code>	<code>ADMIN</code> <code>AUTHOR</code> <code>READER</code>	Rôle unique déterminant les permissions
<code>createdAt</code>	<code>Date (ISO-Date)</code>	Généré à la création	Date de création du compte

Exemple de document

```
{
  "_id": ObjectId("6a5248391e987ba0b1f5fe41"),
  "fullName": "Test Author",
  "email": "author@test.com",
  "password": "$2a$10$N9qo8uL0ickgx2ZMRZoMy.Ig...",
  "role": "AUTHOR",
  "createdAt": ISODate("2026-07-11T13:48:12.000Z")
}
```

2. Collection : articles

Représente un article de blog. L'auteur et les images sont référencés (`authorId`, URLs MinIO) ; les commentaires ne sont pas embarqués (voir le rapport sur la flexibilité et les performances). Les champs `title` et `content` portent un index texte combiné, utilisé pour la recherche full-text (voir section « Gestion des index » du rapport sur la flexibilité et les performances).

Champ	Type BSON	Contrainte	Description
<code>_id</code>	<code>ObjectId</code>	Généré automatiquement	Identifiant unique MongoDB du document
<code>title</code>	<code>String</code>	Requis (index texte, poids 3)	Titre de l'article
<code>content</code>	<code>String</code>	Requis (index texte, poids 1)	Contenu textuel de l'article
<code>authorId</code>	<code>String</code>	Requis (index)	Référence vers <code>_id</code> du document <code>users</code> , auteur de l'article
<code>authorName</code>	<code>String</code>	Requis (dénormalisé)	Copie du nom de l'auteur, capturée à la création, pour éviter un appel supplémentaire à l'affichage
<code>tags</code>	<code>Array<String></code>	Au moins un tag (index multikey)	Liste des mots-clés associés à l'article
<code>images</code>	<code>Array<String></code>	Optionnel	URLs des images stockées sur MinIO
<code>status</code>	<code>String</code> (enum)	DRAFT PUBLISHED	État de publication de l'article
<code>views</code>	<code>Number</code> (int)	Défaut : 0	Compteur de vues de l'article
<code>createdAt</code>	<code>Date</code> (ISO-Date)	Généré à la création	Date de création de l'article
<code>updatedAt</code>	<code>Date</code> (ISO-Date)	Mis à jour à chaque modification	Date de dernière modification

Exemple de document

```
{
  "_id": ObjectId("6a524a7b1e987ba0b1f5fe42"),
  "title": "Mon premier article",
  "content": "Contenu de test sur MongoDB et Spring Boot.",
  "authorId": "6a5248391e987ba0b1f5fe41",
  "authorName": "Test Author",
  "tags": ["mongodb", "spring", "backend"],
  "images": [
```

```
    "http://localhost:9000/blog-images/articles/3f2a-photo.jpg"
  ],
  "status": "PUBLISHED",
  "views": 12,
  "createdAt": ISODate("2026-07-11T13:51:55.952Z"),
  "updatedAt": ISODate("2026-07-11T14:05:10.310Z")
}
```

3. Collection : comments

Représente un commentaire, rattaché à un article via `articleId` (référencement, collection séparée). Le nom de l'auteur est dénormalisé (`authorName` en texte libre) plutôt que référencé vers `users`, conformément au choix de favoriser la disponibilité en lecture propre au NoSQL.

Champ	Type BSON	Contrainte	Description
<code>_id</code>	<code>ObjectId</code>	Généré automatiquement	Identifiant unique MongoDB du document
<code>articleId</code>	<code>String</code>	Requis (index)	Référence vers <code>_id</code> du document <code>articles</code> concerné
<code>authorName</code>	<code>String</code>	Requis	Nom affiché de l'auteur du commentaire (dénormalisé)
<code>content</code>	<code>String</code>	Requis	Contenu du commentaire
<code>createdAt</code>	<code>Date (ISO-Date)</code>	Généré à la création	Date de publication du commentaire

Exemple de document

```
{
  "_id": ObjectId("6a52501e1e987ba0b1f5fe45"),
  "articleId": "6a524a7b1e987ba0b1f5fe42",
  "authorName": "Visiteur",
  "content": "Super article, merci pour le partage !",
  "createdAt": ISODate("2026-07-11T14:12:03.221Z")
}
```

4. Relations entre collections

Le schéma ci-dessous résume les relations de référencement entre les trois collections :

```
users (1) -----< articles      (via articles.authorId)
articles (1) ---< comments      (via comments.articleId)
```

Les relations structurelles entre entités (article vers auteur, commentaire vers article) sont modélisées par référencement, et non par embarquement complet d'un document dans un autre : chaque collection reste indépendante et interrogeable individuellement. Ce référencement est toutefois complété par une dénormalisation ciblée de certains champs de lecture fréquente (`Article.authorName`, `Comment.authorName`), dupliqués volontairement pour éviter des requêtes supplémentaires à l'affichage. Ce choix n'est pas un détail secondaire : il traduit un principe de modélisation propre aux bases NoSQL orientées documents, où la structure des données est pensée en fonction des patterns d'accès de l'application plutôt que d'une normalisation systématique héritée du relationnel.